The `RequestRateLimiter` filter relies on a `KeyResolver` bean to determine which bucket to use for each request. By default, it uses the currently authenticated user in Spring Security. Until we add security to Edge Service, we'll define a custom `KeyResolver` bean and make it return a constant value (for example, `anonymous`) so that all requests will be mapped to the same bucket.

In your Edge Service project, create a `RateLimiterConfig` class in a new `com.polarbookshop.edgeservice.config` package, and declare a `KeyResolver` bean, implementing a strategy to return a constant key.

Listing 9.15 Defining a strategy to resolve the bucket to use per request

```
package com.polarbookshop.edgeservice.config;

import reactor.core.publisher.Mono;
import org.springframework.cloud.gateway.filter.ratelimit.KeyResolver;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RateLimiterConfig {

   @Bean
   public KeyResolver keyResolver() {
      return exchange -> Mono.just("anonymous");     ❶
   }
}
```

❶ Rate limiting is applied to requests using a constant key.

Spring Cloud Gateway is configured to append headers with details about rate-limiting to each HTTP response, which we can use to verify its behavior. Rebuild and run Edge Service (`./gradlew bootRun`), and then try calling one of the endpoints.

```
$ http :9000/books
```